

PathFinder Scholarship Matcher

Weekly Mini Project 02 — Team 1 · Project Specification

Project	PathFinder Scholarship Matcher
Team leader	Shiam Ezzo
Team members	Laith Haj Hosin, Ali Mikdad, Fadi Habil, Mustafa Al Shaar, Rami Al Shaar

Help a student discover which scholarships genuinely fit them — ranked, explained and easy to compare.

1. Project Overview

Students who want to study abroad face a long list of scholarships, each with its own conditions: age limits, education level, field of study, language certificates, countries and funding. Reading every scholarship one by one is slow, and it is easy to miss a condition that makes a scholarship impossible.

PathFinder Scholarship Matcher lets a student describe themselves once, then shows every available scholarship ranked from best match to weakest match, with a match percentage and a clear explanation of why each scholarship fits — or does not.

This is **not** an AI project. The application works with a pre-defined collection of scholarships, and the matching is decided by rules your team designs.

2. Team

The team leader and members are listed in the table at the top of this document.

Team Responsibility

- The leader coordinates the internal work split and makes sure the team reaches a working integrated result.
- Frontend/backend ownership is not assigned by the instructor.
- The team should decide how responsibilities are distributed.
- Every member must make a meaningful contribution.
- Every member should understand the project at a high level.
- Every member must participate in the final English presentation.

3. Project Goal

Build a web application where a student can:

- browse and filter a collection of scholarships;
- fill in a personal study profile;
- receive a ranked list of scholarships with a match percentage for each one;
- understand exactly which requirements they meet and which they are missing;
- save the scholarships they are interested in as favorites.

For the same student profile and scholarship data, the application must always produce one consistent match result.

4. Technology Constraints

Area	Requirement
Frontend	React + TypeScript
Backend	NestJS
Data	In-memory only
Database	Not required

- The application's working data is kept in memory and may reset when the application server restarts.
- How you organize your application is your team's decision.
- Do **not** add a database, user accounts/login, external AI services, cloud services or paid APIs.

5. User Scenario

Lina is 22 and has just finished her bachelor's degree in Computer Science. She speaks Arabic and English (she has an IELTS-level certificate), she cannot afford to pay tuition herself, and she would prefer to study in Germany or Canada.

She opens PathFinder, fills in her profile and asks for her matches. The application shows a ranked list: a fully funded master's scholarship in Germany for Computer Science sits at the top with a high match percentage. A scholarship that requires French appears much lower, clearly marked with "Missing requirement: French language". A PhD-only scholarship is shown as not eligible, because she does not yet hold a master's degree.

She opens the top scholarship, reads why it matches, checks the deadline and saves it to her favorites so she can return to it later.

6. Required Features

Feature	Description
Browse scholarships	View all available scholarships with their key information.
Scholarship details	Open one scholarship and see its full information.
Filter scholarships	Narrow the scholarship list by country, field of study, education level and funding level. Filters can be combined.
Student profile	Fill in and edit a student profile (see the profile information below).
Match calculation	Calculate a match percentage for every scholarship against the current profile.
Ranked results	Show matches ordered from best match to weakest match, with a visible ranking position.
Match explanation	For each result, show which requirements are met and which are missing.
Favorites	Save a scholarship as a favorite and remove it again.
Favorites list	View all saved favorites in one place.

Scholarship information

Each scholarship must present at least the following information to the user:

- title
- country
- university / institution

- eligible fields of study
- required education level
- language requirements
- age restriction (if the scholarship has one)
- funding level (for example: fully funded, partially funded, tuition only)
- application deadline
- short description

Student profile information

The profile must capture at least:

- age
- current field of study
- current education level
- languages the student can use
- budget / financial capacity
- preferred countries
- preferred study areas

7. Business Rules

Eligibility and matching

1. Every scholarship receives a match percentage between **0%** and **100%** for the current profile.
2. Some scholarship conditions are **eligibility requirements** (for example: age limit, required education level, required language, eligible fields). Others are **preferences** (for example: preferred country, preferred study area, funding fit).
3. A scholarship that fails an eligibility requirement must be clearly marked **Not eligible** and must **never** rank above a scholarship for which the student is eligible.
4. An obviously ineligible scholarship must not receive a high match percentage.
5. Among eligible scholarships, those that also fit more of the student's preferences must rank higher.
6. The student's financial capacity must influence the result: a student with a very limited budget must see well-funded scholarships ranked above scholarships that leave large costs to the student.
7. When two scholarships have the same match percentage, the one with the **earlier** application deadline appears first.
8. Scholarships whose application deadline has already passed must not appear in the match results as available opportunities. (They may still be viewed in the scholarship list, clearly marked as closed.)
9. Every result must be **explainable**: the user must be able to see which conditions were met and which were missing. The explanation must agree with the percentage — a result cannot say "all requirements met" and show a low score.
10. A scholarship's match percentage, eligibility status and explanation must be the same wherever it appears in the application.

Consistency

1. The same profile must always produce the same results and the same order.
2. Filters and matching must agree: applying a filter to the results must only hide results — it must never change a scholarship's percentage or explanation.
3. Changing the profile must update the match results the next time they are shown.

Favorites

1. A scholarship can only be in the favorites list once.
2. Removing a favorite removes it from the favorites list immediately.
3. Favorites remain saved while the application is running, including after a page refresh.

8. Required Screens / Views

Screen	Must show / allow
Scholarships	All scholarships with title, country, institution, funding level and deadline; the filters; a clear "closed" marker for expired scholarships.
Scholarship Details	All scholarship information; an option to add to / remove from favorites. If a profile exists, the match percentage and explanation for this scholarship.
Student Profile / Matcher	The profile form; the ability to edit an existing profile; an action that produces the match results.
Match Results	The ranked list with ranking position, match percentage, eligibility status and a short explanation per scholarship; the ability to open the details of each result.
Favorites	All saved scholarships with their key information; the ability to remove each one and to open its details. A clear message when there are no favorites yet.

All screens must be reachable through the application's navigation.

9. Required User Flows

Flow A — Explore scholarships

1. The user opens the Scholarships screen.
2. The user selects one or more filters (for example: country = Germany, funding = fully funded).
3. The list shows only scholarships that satisfy every selected filter.
4. The user clears the filters and the full list returns.
5. The user opens a scholarship and reads its details.

Flow B — Create a profile and get matches

1. The user opens the Student Profile screen.
2. The user fills in all the required profile information.
3. The user requests their matches.
4. If the profile is incomplete or invalid, the user sees which information must be corrected.
5. If the profile is valid, the Match Results screen shows every open scholarship ranked from best to weakest, with percentages and explanations.

Flow C — Understand a result

1. On the Match Results screen, the user selects a scholarship that has a low percentage.
2. The application shows which requirements are met and which are missing (for example: "Required language: French — not in your profile").
3. The user understands why this scholarship ranks low without needing to compare it manually.

Flow D — Change the profile

1. The user returns to the Student Profile screen and changes one value (for example, adds a new language).
2. The user requests their matches again.

- Scholarships that required that language now show the requirement as met, and the ranking changes accordingly.

Flow E — Manage favorites

- From the Scholarship Details or Match Results screen, the user adds a scholarship to favorites.
- The user opens the Favorites screen and sees it listed.
- The user removes it, and it disappears from the list.
- Trying to add the same scholarship twice does not create a duplicate.

10. Required Calculations / Derived Information

The application must produce and display:

Derived information	Where it appears
Match percentage (0–100%) per scholarship	Match Results, Scholarship Details
Ranking position (1st, 2nd, 3rd ...)	Match Results
Eligibility status (Eligible / Not eligible)	Match Results, Scholarship Details
Matching reasons — the conditions the student satisfies	Match Results, Scholarship Details
Missing eligibility requirements — the conditions the student does not satisfy	Match Results, Scholarship Details
Number of eligible scholarships for the current profile	Match Results
Deadline status (open / closed)	Scholarships, Scholarship Details

How the percentage is calculated is your team's design decision, but it must respect every rule in section 7 and your team must be able to explain it in the presentation.

11. Validation & Error Scenarios

General expectations

- Required information cannot be missing.
- Numeric values must be sensible (no negative, zero or absurd values where they make no sense).
- Invalid actions must be rejected — the system must never silently accept an invalid operation.
- The user must always receive a clear, human-readable message explaining **why** an action failed.
- Invalid operations must be rejected even if the user reaches them through an unexpected interaction (for example, an outdated page or a repeated submission).
- When a request cannot be completed or something goes wrong, the screen must show a sensible message — never a blank page or a frozen screen.

Project-specific scenarios

Situation	Expected behavior
The user requests matches without completing the profile	Rejected; the user is told which profile information is missing.
Age is missing, negative, zero or unrealistic (for example 3 or 150)	Rejected with a clear message.
Budget / financial capacity is negative or not a number	Rejected with a clear message.
No language, no preferred country or no field of study selected	Rejected; the user is told what is required.

Situation	Expected behavior
The user opens a scholarship that does not exist	A clear "scholarship not found" message, not a broken page.
The user adds a scholarship that is already a favorite	No duplicate is created; the user is informed it is already saved.
The user removes a scholarship that is not in favorites	Rejected or handled gracefully with a clear message.
Filters match no scholarship	A clear "no scholarships match these filters" message.
The profile is valid but no scholarship is eligible	The results still show, ranked, with a clear message that none are fully eligible.

12. In-Memory Data Requirements

No database is required.

- The application may start with seeded (pre-loaded) data.
- Data created or changed while the application is running must behave normally during that run — every screen must reflect the latest data, including after a page refresh.
- It is acceptable that all data resets when the application server restarts. This is an expected limitation of this assignment — do not spend time adding any other persistence technology.

For this project specifically:

- The application must start with at least **15 realistic scholarships**, covering at least **5 countries**, at least **3 education levels**, several fields of study, different language requirements and different funding levels.
- The seeded collection must include at least one scholarship with an **age restriction**, at least one with an **expired deadline**, and at least one that a typical profile is **not eligible** for — so every rule can be demonstrated.
- Scholarship content is written by your team. It may be inspired by real scholarships but does not need to be real or current.

13. Out of Scope

The following are **not required** and should not be built as part of the core project: real AI or external AI services; scraping websites or external scholarship APIs; user accounts, login or registration; a database; applying to a scholarship through the application.

14. Minimum Deliverable

Your project counts as complete only when **all** of the following work:

- ☐ A working React + TypeScript frontend.
- ☐ A working NestJS backend.
- ☐ The frontend and backend work together as one integrated application; every change a user makes is visible on all related screens.
- ☐ Every required screen in section 8 exists and is reachable through the application's navigation.
- ☐ Every business rule in section 7 is enforced.
- ☐ Invalid operations are rejected with useful, clear feedback.
- ☐ In-memory data changes correctly while the application is running.
- ☐ The project can be demonstrated from start to finish without manual fixes during the demo.
- ☐ Scholarships can be browsed, filtered (with combined filters) and opened in detail.
- ☐ A complete profile can be created and edited.

- ☐ Every open scholarship receives a match percentage and an explanation for the current profile.
- ☐ Results are ranked correctly, with ineligible scholarships below eligible ones.
- ☐ Changing the profile changes the results.
- ☐ Favorites can be added, viewed and removed, without duplicates.

15. Optional Bonus Features

Bonus features are **optional**. They are not needed for full core completion and they never replace a missing core requirement. Only start a bonus after the minimum deliverable is complete and working. Bonus features must not add a database, login or new major technologies.

- **Extra sorting** — sort results by deadline or funding level in addition to match percentage.
- **Advanced filters** — filter the match results by minimum match percentage or "eligible only".
- **Side-by-side comparison** — compare two scholarships and their match explanations on one screen.
- **Improved explanation** — show which single change to the profile would improve a scholarship's match the most.

16. Presentation Expectations

Your team will present the finished project **in English**. Your instructor will confirm the date and the time limit.

The presentation must include:

1. **The problem** — a brief explanation of who the product is for and what problem it solves.
2. **Product walkthrough** — a short tour of the main screens.
3. **Live demonstration** — the main user flows working live, including at least one invalid case being rejected with a clear message.
4. **Business rules** — an explanation, in your own words, of the most important rules your product enforces and how a user experiences them.
5. **Team** — every member speaks.

Before presenting, make sure the backend and frontend both start cleanly and the demo data is ready.

17. Evaluation Focus

You will be assessed on:

Area	What we look for
Functional completeness	The required features and screens exist and work.
Correct business behavior	The business rules are enforced exactly as described.
Frontend/backend integration	The frontend and backend genuinely work together as one working application.
User experience	Screens are clear, readable and easy to use without explanation.
Handling invalid cases	Invalid input and invalid actions are rejected with clear messages.
Team collaboration	Work is shared; every member contributed meaningfully.
Code organization and readability	Your code is understandable and reasonably organized.
Explaining your own work	Every member can explain the product and its rules.
Presentation quality	Clear English presentation with a working live demo.